Poster Project\PosterProject.py

```python
import pandas as pd
import numpy as np

data1 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A778_D0_gene.tsv",sep="\t")
data2 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A778_D6_gene.tsv",sep="\t")
data3 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A820_D0_gene.tsv",sep="\t")
data4 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A820_D6_gene.tsv",sep="\t")
data5 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A870_D0_gene.tsv",sep="\t")
data6 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A870_D6_gene.tsv",sep="\t")
data7 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A899_D0_gene.tsv",sep="\t")
data8 = pd.read_csv("Poster Project\\Datasets\\raw\\1_CancerTranscriptomics\\read_counts\\A899_D6_gene.tsv",sep="\t")

file = ["A778_D0_gene","A778_D6_gene",
        "A820_D0_gene","A820_D6_gene",
        "A870_D0_gene","A870_D6_gene",
        "A899_D0_gene","A899_D6_gene"]

data_sets = [data1, data2, data3, data4, data5, data6, data7, data8]
headers = ["Ensembl Gene Record","Common Gene Name","Read Count"]

'''DATA CLEANING'''
# adding headers for initial datasets
# remove QC values from datasets, adding log correction and visualizing initial data structure
for i, data in enumerate(data_sets):
    data = data.iloc[:-5] # removes “__no_feature”."__ambiguous”.”__too_low_aQual”,”__not_aligned”,”__alignment_not_unique”
    data.columns = headers
    mask = data['Read Count'] != 0
    data.loc[mask, 'Log Count'] = np.log(data.loc[mask, 'Read Count']) #performing log correction for visualization later, igonre counts that equal 0
    data_sets[i] = data
    data.to_csv("Poster Project\\Datasets\\1_CancerTranscriptomics\\read_counts\\"+file[i]+".tsv", index = False,sep="\t")
    # print(data.info())
    # print(data.describe())

# combining .tsv files for future data processing
combinedData = data_sets[0].drop(columns=['Common Gene Name','Log Count'])
for i, data in enumerate(data_sets[1:]):
    combinedData=pd.concat([combinedData,data['Read Count']],axis=1)

headers = [headers[0]]+file
combinedData.columns=headers

# Pre-filtering -> removing genes that have low read count (total feature count of less than 10)
combinedData = combinedData[combinedData.iloc[:, 1:].sum(axis=1)>=10]
combinedData.to_csv("Poster Project\\Datasets\\combinedData.tsv",index=False,sep='\t')

#combining Day0s and Day6s for future data processing
Day0data = combinedData.drop(columns=file[1::2])
Day6data = combinedData.drop(columns=file[0::2])

Day0data.to_csv("Poster Project\\Datasets\\Day0.tsv", index = False,sep="\t")
Day6data.to_csv("Poster Project\\Datasets\\Day6.tsv", index = False,sep="\t")

'''VISUALIZING THE DISTRIBUTION OF COUNTS PER GENE'''
# Log correction reveals that the count distribution is bimodal, suggesting that there are two distributions
# Genes that are upregulated and genes that are down regulated with significant overlap
# Genes are seperated based on whether the expression of gene is clearly up or down regulated
bin_numb=40

def binDist(data:pd.DataFrame)->tuple:
    bins = {}
    ran = float(data["Log Count"].max()-data["Log Count"].min())
    inc = ran/bin_numb
    for j in range(bin_numb):
        bins[(j*inc,(j+1)*inc)]=0
```

```
    for val in data["Log Count"]:
        for bin in bins:
            if val > bin[0] and val < bin[1]:
                bins[bin] +=1
    avg = sum(bins.values())/len(bins)
    listr = []
    for value in bins.values():
        listr.append(value)
    std = np.std(listr)
    return (avg,std)

for i, data in enumerate(data_sets):
    data["Dist"] = pd.cut(data['Log Count'], bins=bin_numb, labels=list(range(bin_numb))) #distributing genes based on log Count
    data["Dist"] = data["Dist"].fillna(0).astype(str).astype(int)
    data['Dist'] = data['Dist'].astype(str).astype(int)
    # data['Dist'] = data['Dist'].replace(0, np.nan)
    counts, bin_edges = np.histogram(data['Dist'], bins=bin_numb)

    avg,std = binDist(data)
    first, last = (0,0)
    for count in counts[2:]:    #selecting bins based on count density in one standard deviation from suppressed to expressed peak
        if count < avg+std:
            first = list(np.where(counts == count))
            first = first[0][0]
            for count in counts[first:]:
                if count > avg+std:
                    last = np.where(counts == count)
                    last = last[0][0]
                    break
            break


    NormalData = data[(data["Dist"]>first)&(data["Dist"]<last)] #NormalData contains all genes that have counts that are between one standard deviation of counts
    ExtrmeData = data[(data["Dist"]<first)|(data["Dist"]>last)] #ExtremeData contains all genes that have counts that are outside one standard deviation of counts
    # print(NormalData.describe())
    # print(ExtrmeData.describe())
    NormalData.to_csv("Poster Project\\Datasets\\NORMAL\\NORMAL_"+file[i]+".tsv", index = False,sep="\t")
    ExtrmeData.to_csv("Poster Project\\Datasets\\EXTREME\\EXTREME_"+file[i]+".tsv", index = False,sep="\t")
```